

Evaluación Parcial N° 2

Implementación de Módulos Terraform Estudiante

Sigla	Nombre Asignatura	Tiempo Asignado	% Ponderación
AUY1105	Infraestructura como código II	5 horas pedagógicas	30%

1.Situación Evaluativa:

X	Ejecución práctica
---	--------------------

	Entrega de encargo
--	-----------------------

	Presentación
--	--------------

2.Instrucciones

Descripción general de la evaluación

El propósito de esta evaluación es desarrollar módulos de Terraform que sigan buenas prácticas en términos de documentación, versionado y reutilización de código. Los estudiantes deberán desacoplar el código trabajado en la Evaluación Parcial 1 en módulos independientes que permitan mayor escalabilidad y reutilizabilidad. La evaluación incluye la configuración de repositorios de código en GitHub, la creación de módulos bien estructurados, y la aplicación de estándares de versionado semántico.

- El propósito de esta evaluación es evaluar los siguientes Indicadores de Logro:

- IL3.1 Desarrolla módulos de Terraform reutilizables siguiendo las mejores prácticas de codificación y estructura, asegurando que sean modulares, eficientes y de fácil integración en distintos entornos.
 - IL3.2 Genera documentación completa y clara para cada módulo, incluyendo ejemplos prácticos de uso, descripciones detalladas de los parámetros configurables y posibles dependencias, facilitando su implementación.
 - IL3.3 Implementa un sistema de versionado semántico para los módulos publicados, gestionando adecuadamente las actualizaciones y asegurando compatibilidad hacia atrás, con el fin de minimizar riesgos de ruptura y facilitar la evolución del código.
- Esta evaluación consiste en una ejecución práctica donde deberá tomar implementar módulos de terraform, usando buenas prácticas de documentación y versionamiento.
 - **Tiempo** asignado para esta evaluación es de 5 *horas pedagógicas* y se realiza en parejas en *laboratorio TAITE 09*.

Instrucciones Específicas

El propósito de esta evaluación es desarrollar módulos de Terraform que sigan buenas prácticas en términos de documentación, versionado y reutilización de código. Los estudiantes deberán desacoplar el código trabajado en la Evaluación Parcial 1 en módulos independientes que permitan mayor escalabilidad y reutilizabilidad. La evaluación incluye la configuración de repositorios de código en GitHub, la creación de módulos bien estructurados, y la aplicación de estándares de versionado semántico.

Todo el trabajo realizado deberá registrarse en los repositorios Github asignados. Cada cambio, configuración o ajuste deberá ser subido y versionado mediante commits claros y descriptivos. Cada participante deberá trabajar utilizando su usuario personal de GitHub. No se aceptarán contribuciones que no puedan ser identificadas con un usuario personal. Las parejas asignadas al inicio del parcial 1 se mantendrán de forma constante durante el parcial 2.

La evaluación se divide en tres grandes apartados que deben ser evidenciados detalladamente en los repositorios de GitHub.

1. Repositorios de Código

Se deben **crear tres repositorios** en GitHub para almacenar el código y el flujo de automatización asociado. La estructura requerida es la siguiente:

- **Repositorio Principal**

Se utilizará el repositorio de la *Evaluación Parcial 1* como base para almacenar el código principal de la infraestructura. Este código será utilizado para llamar y ejecutar los módulos creados.

- **Repositorio Módulo Redes**

Contendrá el módulo de Terraform para redes, encargado de gestionar la creación de VPC, subnets y security groups.

- **Nombre sugerido:** terraform-aws-vpc-<sigla-curso>-grupo-<Nº de grupo>

- **Repositorio Módulo Cómputo**

Contendrá el módulo de Terraform para la configuración de recursos de cómputo, como instancias EC2.

- **Nombre sugerido:** terraform-aws-ec2-<sigla-curso>-grupo-<Nº de grupo>

Configuración Básica de los Repositorios

Cada repositorio deberá incluir lo siguiente:

- **Descripción del repositorio:** Breve pero clara sobre los objetivos del repositorio.
- **Archivo README.md:** Debe incluir:
 - Objetivos del repositorio.
 - Propósito general del código Terraform.
 - Instrucciones básicas de uso.
- **Archivo .gitignore:** Excluir archivos innecesarios como .terraform/, secretos y claves privadas.
- **Archivo CHANGELOG.md:** Documentar cambios realizados en cada versión del proyecto.
- **Archivos Terraform:** Cada módulo debe incluir al menos los siguientes archivos:
 - main.tf: Definición principal del módulo.
 - variables.tf: Declaración de variables con descripciones claras.
 - outputs.tf: Declaración de outputs con descripciones claras.
 - versions.tf: Declaración de las versiones requeridas de Terraform y los proveedores que se emplearán en el proyecto.

2. Código de Infraestructura

2.1. Módulos

En esta evaluación, se desarrollarán dos módulos específicos: uno para redes y otro para cómputo. Estos módulos deben estar diseñados siguiendo buenas prácticas para garantizar su claridad, reutilización y mantenimiento a largo plazo.

Objetivos de los Módulos

1. Facilitar la reutilización del código al desacoplar componentes específicos de la infraestructura.
2. Permitir la fácil configuración e integración de los módulos en otros proyectos.
3. Asegurar la calidad del código mediante buenas prácticas de documentación, estructura y pruebas.

El código de la evaluación parcial 1 debe adaptarse para construir dos módulos reutilizables, detallados a continuación:

- **Módulo de Redes**
 - Código necesario para la creación de VPC, subnets y security groups.
 - Variables: El módulo debe estar parametrizado en todas las variables necesarias para su implementación.
 - Outputs: Se deben configurar cómo mínimo outputs para `vpc_id` y `subnet_ids`.
- **Módulo de Cómputo**
 - Incluye la definición de recursos EC2 y otros elementos necesarios para su despliegue.
 - Variables: El módulo debe estar parametrizado en todas las variables necesarias para su implementación.
 - Outputs: Se deben configurar cómo mínimo outputs para `instance_id` e `instance_ip`.

Buenas prácticas generales para implementar:

- Estructura de módulos: Cada módulo debe seguir el estandar y buenas prácticas descritas por [terraform](#). Como mínimo los módulos deben contener los siguientes archivos.
 - main.tf: Definición principal del módulo.
 - variables.tf: Declaración de variables con descripciones claras.
 - outputs.tf: Declaración de outputs con descripciones claras.
 - versions.tf: Declaración de las versiones requeridas de Terraform y los proveedores que se emplearán en el proyecto.
- Carpeta examples/:
 - Archivo README.md: Explicación del objetivo y uso del módulo.
 - Ejemplos funcionales de uso del módulo.

2.2. Terraform Principal

El repositorio principal servirá como punto de referencia para consolidar y coordinar la implementación general de la infraestructura desarrollada. Este repositorio debe incluir tanto los componentes que integran los módulos desacoplados como los flujos de trabajo necesarios para desplegar y gestionar la infraestructura.

Objetivos del Repositorio Principal

1. Actuar como un controlador central que orqueste los módulos desacoplados desarrollados en los repositorios de redes y cómputo.
2. Almacenar configuraciones de alto nivel para la infraestructura completa, permitiendo la integración de los módulos de forma coherente.
3. Servir como base para la automatización del despliegue y pruebas de la infraestructura.

Automatización

Se deben mantener y adaptar en caso de ser necesario los pasos de validación de código estático y seguridad desarrollados en la evaluación Parcial 1.

Documentación

Se debe actualizar la documentación archivo README, CHANGELOG reflejando los cambios realizados.

3. Versionado Módulos

El versionado es un componente clave para garantizar la trazabilidad, consistencia y control de los cambios en el código. Para esta evaluación, se deben seguir las **buenas prácticas de versionado semántico (Semantic Versioning)**, que consiste en identificar las versiones del código mediante un formato de tres números: MAJOR.MINOR.PATCH. **Se deben cumplir los siguientes requisitos detallados a continuación:**

- **Etiqueta de la Versión Inicial**

Al finalizar la implementación del módulo, se debe crear una etiqueta (*release tag*) para indicar la versión inicial del código, v1.0.0. Esta etiqueta debe reflejar un estado estable y funcional del módulo. Para versiones previas a la inicial estas deben estar identificadas por una etiqueta inferior a la versión v1.0.0, empezando desde la versión v0.1.0.

- **Documentación en CHANGELOG.md**

Registrar cada cambio realizado en el módulo en un archivo llamado CHANGELOG.md. Este archivo debe tener un formato claro y estandarizado. Se debe registrar número de versión, fecha de modificación, y el tipo de cambio realizado **Added** (Agregado), **Changed** (Modificado), **Fixed** (Corregido), o **Removed** (Eliminado).

- **Control de Versiones en GitHub**

Usar las herramientas de GitHub para gestionar las versiones:

- Crear un *release* con un nombre correspondiente al número de versión (por ejemplo, v1.0.0).
- Incluir una descripción en el *release* que resuma los cambios realizados.

Asociar cada versión con un commit específico mediante las etiquetas (*tags*).

Materiales e insumos

- Laboratorio TAITE 09
- Conectividad a Internet.

- Acceso a una nube pública.

Aspectos Formales

Producto: Evidenciar el paso a paso **detallado** en los enlaces al repositorio de código Github.

3.Pauta de Evaluación

Tipo de Pauta: (Rúbrica, Escala de Valoración, Pauta de cotejo)

Categoría	% logro	Descripción niveles de logro
Muy buen desempeño	100%	Demuestra un desempeño destacado, evidenciando el logro de todos los aspectos evaluados en el indicador.
Buen desempeño	80%	Demuestra un alto desempeño del indicador, presentando pequeñas omisiones, dificultades y/o errores.
Desempeño aceptable	60%	Demuestra un desempeño competente, evidenciando el logro de los elementos básicos del indicador, pero con omisiones, dificultades o errores.
Desempeño incipiente	30%	Presenta importantes omisiones, dificultades o errores en el desempeño, que no permiten evidenciar los elementos básicos del logro del indicador, por lo que no puede ser considerado competente.
Desempeño no logrado	0%	Presenta ausencia o incorrecto desempeño.

Indicador de Evaluación	Categorías de Respuesta					Ponderación Indicador de Evaluación
	Muy buen desempeño 100%	Buen desempeño 80%	Desempeño aceptable 60%	Desempeño incipiente 30%	Desempeño no logrado 0%	
IE3.1.1 Desarrolla módulos de Terraform reutilizables siguiendo las mejores prácticas de codificación y estructura, asegurando que sean modulares, eficientes y de fácil integración en distintos entornos.	Diseña módulos de Terraform altamente modulares, eficientes y bien estructurados, siguiendo las mejores prácticas de codificación.	Desarrolla módulos de Terraform efectivos, con buena estructura y adherencia a las mejores prácticas. Son bastante modulares y se integran en	Crea módulos de Terraform funcionales y reutilizables, pero con ciertas limitaciones en modularidad o eficiencia. Requieren ajustes para una integración óptima en	Diseña módulos de Terraform básicos que cumplen parcialmente las mejores prácticas. Tienen baja modularidad y eficiencia, dificultando su	No diseña módulos de Terraform.	33%

	Los módulos son reutilizables y se integran fácilmente en distintos entornos.	varios entornos con mínimas adaptaciones.	algunos entornos.	integración en otros entornos.		
IE3.2.1 Genera documentación completa y clara para cada módulo, incluyendo ejemplos prácticos de uso, descripciones detalladas de los parámetros configurables y posibles dependencias, facilitando su implementación.	Documentación completa y clara para cada módulo, con ejemplos prácticos detallados, descripciones exhaustivas de todos los parámetros configurables y dependencias. Facilita la implementación de manera sobresaliente.	Documentación detallada y útil, con ejemplos prácticos y descripciones de la mayoría de los parámetros y dependencias. La implementación es clara, aunque podría incluir algunos detalles adicionales.	Documentación funcional y útil, pero básica. Incluye ejemplos y describe los parámetros principales, pero faltan detalles en algunas dependencias, lo que limita la facilidad de implementación.	Documentación limitada, con pocos ejemplos y descripciones generales. Algunos parámetros y dependencias no están explicados, dificultando la implementación.	Documentación insuficiente o inexistente, sin ejemplos prácticos ni descripciones útiles.	33%
IE3.3.1 Implementa un sistema de versionado semántico para los módulos publicados, gestionando adecuadamente las actualizaciones y asegurando compatibilidad hacia atrás, con el fin de minimizar riesgos de ruptura y facilitar la evolución del código.	Implementa y gestiona el versionado semántico con precisión. Minimiza riesgos de ruptura y facilita una evolución estable y segura del código.	Implementa y gestiona el versionado semántico. Aunque no es totalmente consistente, minimiza riesgos de ruptura y facilita una evolución estable y segura del código.	Aplica el versionado semántico, con una gestión de actualizaciones y una compatibilidad básica.	Implementa un sistema de versionado semántico mínimo, con gestión limitada de actualizaciones y problemas frecuentes de compatibilidad.	No implementa un sistema de versionado semántico adecuado.	34%

Total	100%
-------	------